developer.nvidia.com

An Easy Introduction to Multimodal Retrieval-Augmented Generation

Annie Surla

13–16 minutes

A retrieval-augmented generation (RAG) application has exponentially higher utility if it can work with a wide variety of data types—tables, graphs, charts, and diagrams—and not just text. This requires a framework that can understand and generate responses by coherently interpreting textual, visual, and other forms of information.

In this post, we discuss the challenges of tackling multiple modalities and approaches to build a multimodal RAG pipeline. To keep the discussion concise, we focus on just two modalities, image and text. Check out our related post, An Easy Introduction to Multimodal Retrieval-Augmented Generation for Video and Audio.

Why is multimodality hard?

Enterprise (unstructured) data is often spread across multiple modalities, whether it's a folder filled with high resolution images or PDFs containing a mix of text tables, charts, diagrams, and so on.

There are two major points to consider when working with this type of modality spread: each modality has its own challenges and how do you manage information across modalities?

Each modality has its own challenges

For instance, consider images (Figure 1). For the image on the left, the focus is more on general imagery rather than minute details. The attention is only on a few key points like the pond, ocean, trees, and sand.

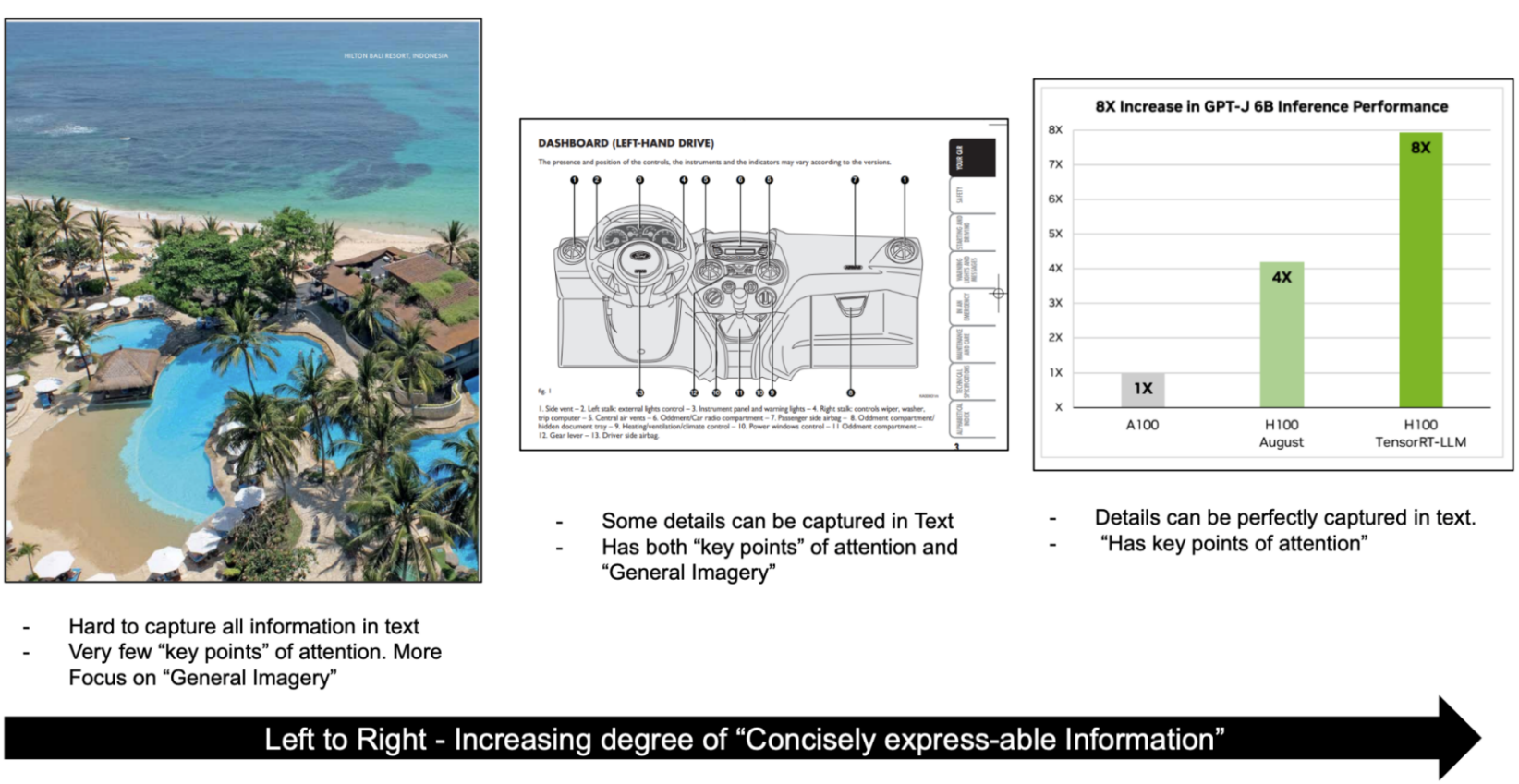


Figure 1. Increasing degree of concisely expressed information in images

Reports and documentations may contain information-dense images, like charts and diagrams, which have many points of interest and additional context that can be derived from the image. Whatever pipeline you build must capture and address these nuances to effectively embed information.

How do you manage information across modalities?

Another crucial aspect is representing information across different modalities. For instance, if you are working with a document, you must make sure that the semantic representation of a chart aligns with the semantic representation of the text discussing the same chart.

Approaches for multimodal retrieval

With the key challenge understood, here are the specifics of building RAG pipelines to tackle these challenges.

There are several main approaches to building multi-modal RAG pipelines:

- Embed all modalities into the same vector space
- Ground all modalities into one primary modality
- Have separate stores for different modalities

To keep this discussion concise, we only discuss images and text input.

Embed all modalities into the same vector space

In the case of images and text, you can use a model like CLIP to encode both text and images in the same vector space. This makes it so that you can largely use the same text-only RAG infrastructure

and swap out the embedding model to accommodate another modality. For the generation pass, you then replace the large language model (LLM) with a multimodal LLM (MLLM) for all question and answering.

This approach simplifies the pipeline, as the only change required in the generic retrieval pipeline is that of swapping the embedding model.

The tradeoff in this situation is to have access to a model that can effectively embed different types of images and text and also capture all the intricacies like text in images and complex tables.

Ground all modalities into one primary modality

Another option is to pick a primary modality based on the focus of the application and ground all other modalities in the primary modality.

For example, say your application revolves mainly around text-based Q&A over PDFs. In this case, you process text normally but for images, you create text descriptions and metadata in the preprocessing step. You also store the images for later use.

In the inference pass, the retrieval then works primarily off of the text description and metadata for the images, and the answer is generated with a mix of LLMs and MLLMs, depending on the type of image retrieved.

The key benefit here is that the metadata generated from the information-rich image is extremely helpful in answering objective questions. This also works around the need for tuning a new model for embedding images as well as building a re-ranker to rank results from across different modalities. The key disadvantages are preprocessing costs and losing some nuance from the image.

Have separate stores for different modalities

Rank-rerank is another approach where you have separate stores for different modalities, query them all to retrieve top-*N* chunks, and then have a dedicated multimodal re-ranker provide the most relevant chunks.

This approach simplifies the modeling process, so that you don't have to align one model to work with multiple modalities. However, it adds complexity in the form of a re-ranker to arrange the now top-*M***N* chunks (*N* each from *M* modalities).

Multimodal models for generation

LLMs are designed to understand, interpret, and generate text-based information. Trained on vast amounts of textual data, LLMs can perform a range of natural language processing tasks, such as text generation, summarization, question-answering, and more.

MLLMs can perceive more than textual data. MLLMs can handle modalities like images, audio, and video, which is often how real-world data is composed. They combine these different data types to create a more comprehensive interpretation of the information, improving the accuracy and robustness of its predictions.

These models can perform a wide range of tasks:

- Visual language understanding and generation
- Multimodal dialogue
- Image captioning
- Visual question answering (VQA)

These are all tasks that a RAG system can benefit from while dealing with multiple modalities. Having

a deeper understanding of how MLLMs work with images and text requires a look at how these models are constructed.

One of the popular subtypes of MLLMs is Pix2Struct, a pretrained image-to-text model that enables semantic understanding of the visual input with its novel pretraining strategy. These models, as the name suggests, generate structured information extracted from the image. For instance, a Pix2Struct model can extract the key information from charts and express it in text.

With that understood, here's how you can build a RAG pipeline.

Building a pipeline for multimodal RAG

To showcase how you can tackle different modalities of data, we walk you through an application indexing multiple technical posts, such as Breaking MLPerf Training Records with NVIDIA H100 GPUs. This post contains complex images that are charts and graphs with rich text, tabular data, and of course, paragraphs.

Here are the models and tools that you need before you can begin handling the data and building a RAG pipeline:

- **MLLM:** Used for image captioning and VQA.
- **LLM:** General reasoning and question answering.
- **Embedding model:** Encoding data into vectors.
- **Vector database:** Store the encoded vectors for retrieval.

Interpreting multimodal data and creating a vector database

The first step for building a RAG application is to preprocess your data and store it as vectors in a vector store so that you can then retrieve relevant vectors based on a query.

With images present in the data, here is a generic RAG preprocessing workflow to work through (Figure 2).

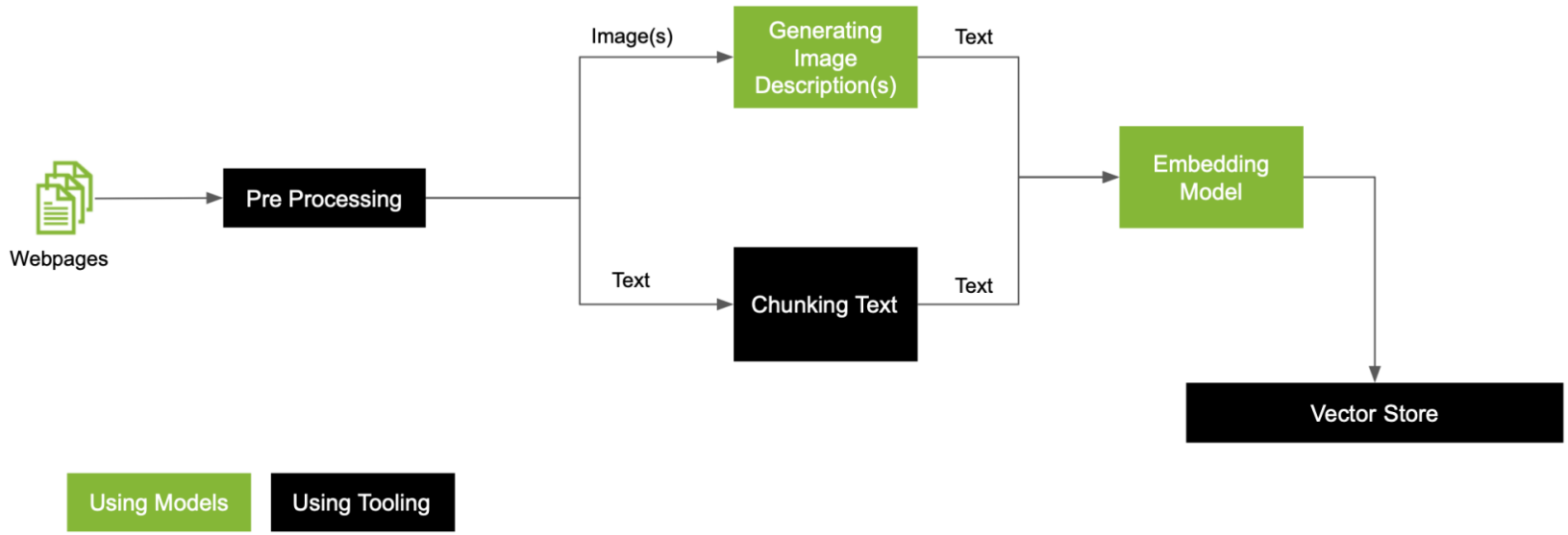
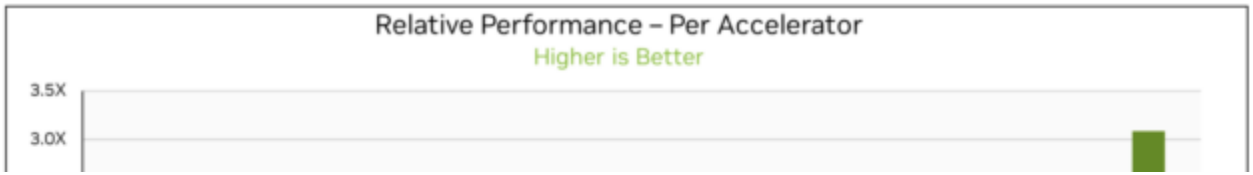


Figure 2. General RAG preprocessing workflow for data with images

The post in question contains several bar charts like Figure 3. To interpret these bar charts, use Google's DePlot, a visual-language model capable of comprehending charts and plots when coupled with an LLM. The model is available on NGC.

For more information about using the DePlot API in your RAG application, see Query Graphs with Optimized DePlot Model.



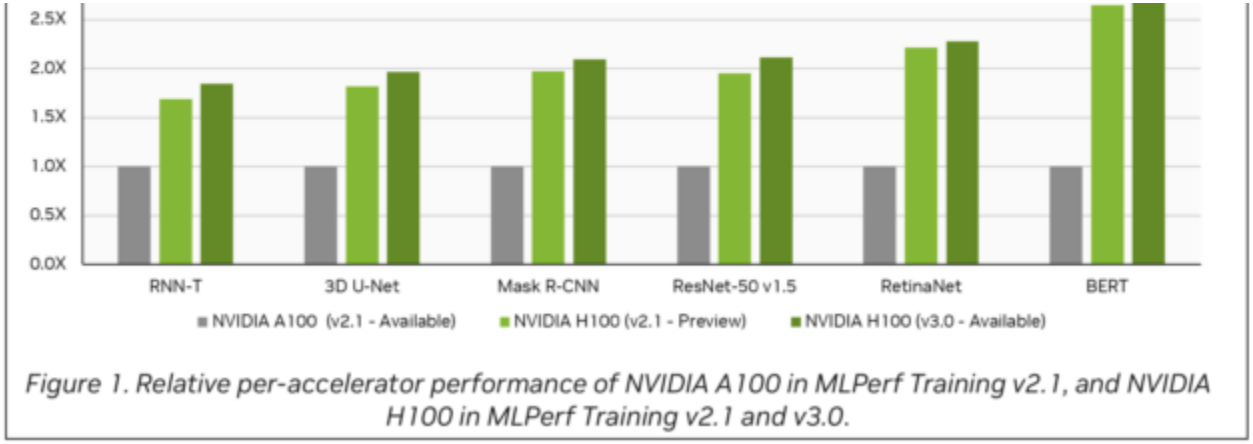


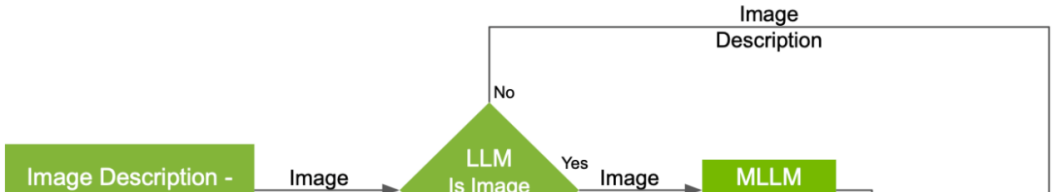
Figure 3. Example bar chart from the PDF, including the caption

This example focuses on charts and plots. Other documents may contain images that may require model customization to handle specialized images, such as medical imagery or schematic diagrams. This depends on the use case but you have several options to tackle this variance in imagery: Either tune one MLLM to handle all types of imagery or to build an ensemble of models for different types of images.

To keep the explanation simple, this is a simple ensemble case with two categories:

- Image with graphs to be processed using DePlot
- Other images to be processed with an MLLM like KOSMOS2

In this post, we expand the preprocessing pipeline to dive deeper into handling each modality in the pipeline that leverages custom text splitters, customized MLLMs, and LLMs to create the VectorDB (Figure 4).



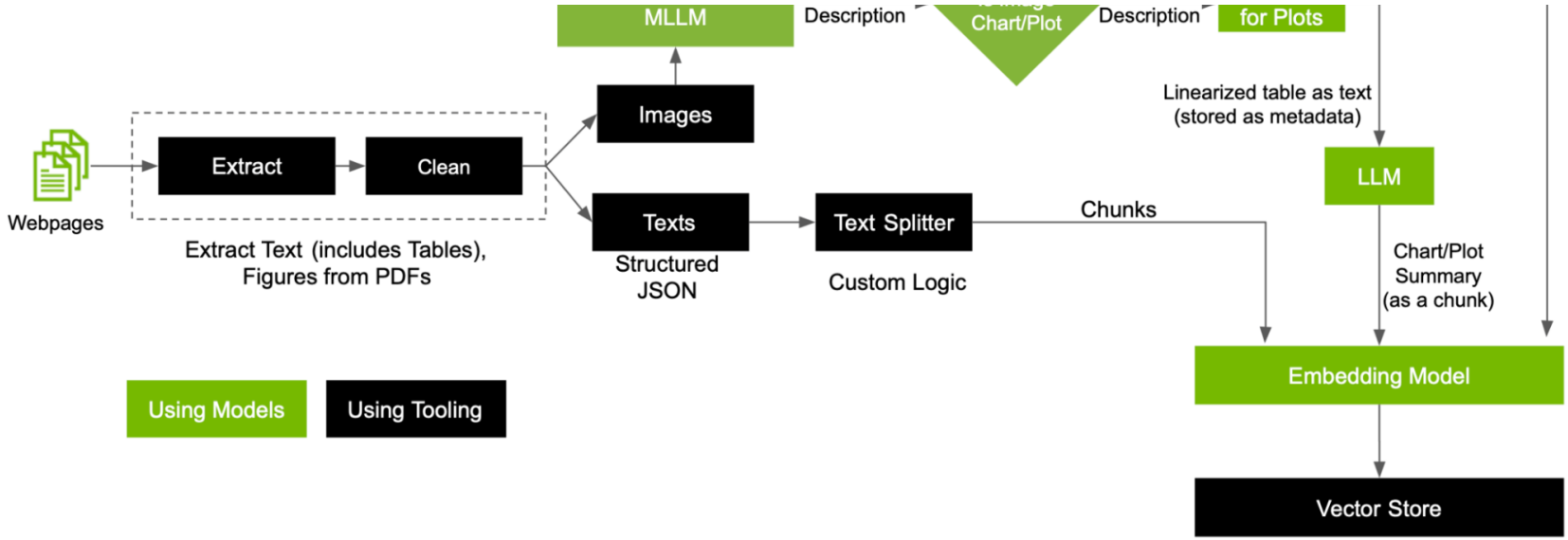


Figure 4. RAG preprocessing workflow with customized MLLMs tackling different types of images

Here are some key steps in the preprocessing workflow:

- Separate images and text
- Classify images using an MLLM based on the image types
- Embed text in PDFs

Separate images and text

The goal is to ground images to text modality. Start by extracting and cleaning your data to separate images and text. You can then go about tackling these two modalities to eventually store them in the vector store.

Classify images using an MLLM based on the image types

Image descriptions generated by an MLLM can be used to classify images into categories whether or not they are graphs. Based on the classification, use DePlot for images containing graphs to generate a linearized tabular text. This text being semantically different from regular text poses a challenge for retrieving the relevant information when performing a search during inference.

We recommend using summaries of the linearized text as chunks to store in the vector store with the outputs from customized MLLMs as metadata, which you can use during inference.

Embed text in PDFs

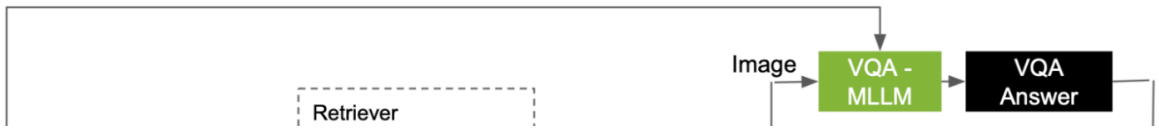
There is room for exploring various text-splitting techniques based on the data that you are working with to achieve the best RAG performance. For simplicity, store each paragraph as a chunk.

Talking to your vector database

Following through this pipeline, you can successfully capture all the multimodal information present in the PDF. Here's how the RAG pipeline works when a user asks a question.

When a user prompts the system with a question, a simple RAG pipeline converts the question into an embedding and performs a semantic search to retrieve some relevant chunks of information. Considering that retrieved chunks also come from images, take a few additional steps before sending all the chunks to the LLM for generating the final response.

Figure 5 shows a reference flow of how to tackle a user query to answer using information retrieved as chunks from both images and text.



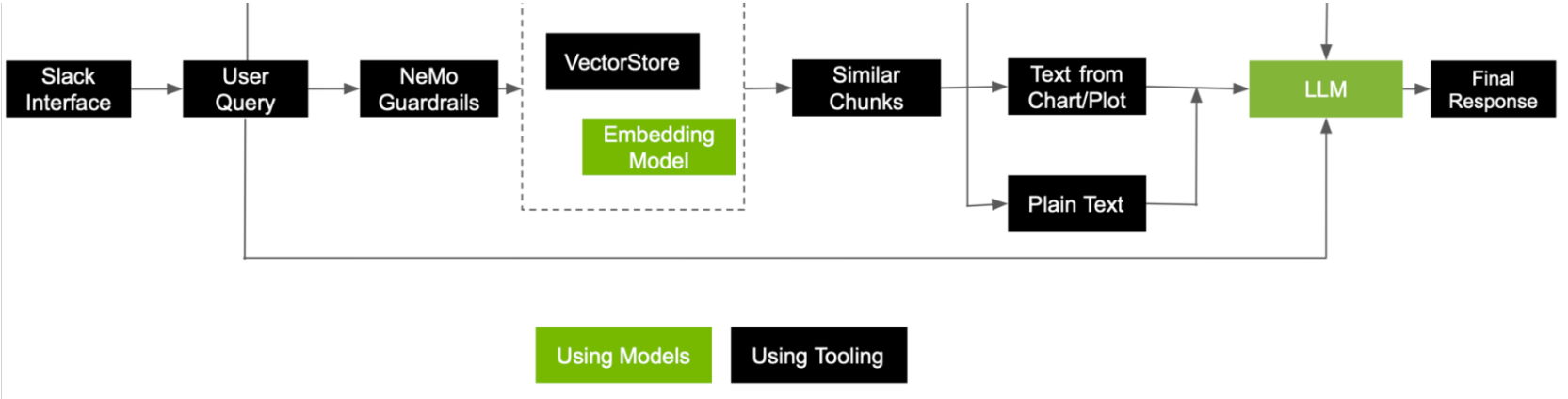


Figure 5. RAG inference workflow with customized MLLMs retrieving Information from multimodal data

Here's an example question prompting a multimodal RAG-enabled bot that has access to the PDF of interest, "What is the difference in performance between NVIDIA A100 and NVIDIA H100(v2.1) with 3D U-Net?"

The pipeline was successful in retrieving the relevant graphical image, and interpreting it with an accurate mention of NVIDIA H100 (v2.1) having a 80% higher relative performance per accelerator than NVIDIA A100 on the 3D U-Net benchmark.

Here are some key steps involved in handling the question after performing a search and retrieving the top five relevant chunks:

- If the chunk was extracted from an image, an MLLM takes the image along with the user question as input to generate an answer. This is nothing but a VQA task. The generated answer is then used as the final context for an LLM to respond.
- If the chunk is extracted from a chart or plot, recall the linearized table stored as metadata and append the text as context to the LLM.
- Finally, the chunks coming from plain text are used as is.

All these chunks, along with the user question, are now ready for the LLM to generate a final answer. From the sources listed in Figure 6, the bot referred to the chart that shows the relative performance on different benchmarks to generate an accurate final response.

Extending the RAG pipeline

This post touches on scenarios where simple text-based questions are answered using data spread across multiple modalities. To further the development of multimodal RAG technology and extend its capabilities, we recommend the following areas of research.

Addressing user questions that include different modalities

Consider a user question consisting of an image containing a graph and a list of questions, what changes in the pipeline are required to accommodate this type of multimodal request?

Multimodal responses

Text-based answers are provided with citations representing other modalities, as in Figure 6. However, a written explanation may not always be the best type of result for a user query. For instance, multimodal responses can be further extended to generate images upon request, such as a stacked bar chart.

Multimodal agents

Solving for complex questions or tasks is beyond simple retrieval. This requires planning, specialized tools, and ingestion engines. For more information, see Introduction to LLM Agents.

Summary

There is much room to improve and explore future multimodal capabilities in generative AI applications, thanks to advancements in multimodal models and an increased demand for RAG-powered tools and services.

Businesses that can integrate multimodal capabilities into their core operations and technology tools are better equipped to scale their AI services and offerings for use cases yet to be listed.

Get hands-on experience with an implementation of the multimodal RAG workflow in GitHub. Or, use NVIDIA NeMo Retriever microservices, including the retrieval embedding model, in the API catalog.